📖 GREENFOTO — The Complete Learning Guide

Understanding Your Own Game, From the Ground Up

> **How to use this guide:** Read it from top to bottom, in order. Each chapter builds on the one before it. Later chapters assume you already understand the earlier ones. Take your time. You built something genuinely impressive here — this guide exists to help you understand what you built.

TABLE OF CONTENTS

1. The Big Picture — What Kind of Game Is This?
2. How Greenfoot Works (The Foundation)
3. Chapter 1 — The Entry Point: `MyWorld.java`
4. Chapter 2 — The Brain: The Game State Manager
5. Chapter 3 — The States: All the "Scenes" of the Game
6. Chapter 4 — The Player: `Player`, `GenericPlayer`, and `Dio`
7. Chapter 5 — The Obstacles: `Obstacles`, `Roadroller`, and `Train`
8. Chapter 6 — The Ability System: The `Ability` Interface
9. Chapter 7 — All the Abilities Explained One by One
10. Chapter 8 — The Time Machine: Rewinding Time
11. Chapter 9 — The Manager Classes (The Specialists)
12. Chapter 10 — The Animation System: `Animator`
13. Chapter 11 — The Characters: `CharacterConfig` and `BossConfig`
14. Chapter 12 — The Configuration Hub: `GameConfig`
15. Chapter 13 — The User Interface (UI Classes)
16. Chapter 14 — The Visual Effects (FX Classes)
17. Chapter 15 — The Interfaces and Contracts
18. Chapter 16 — The Full Data Flow: One Frame of Gameplay, Step by Step
19. Appendix A — The Complete File Map (All 76 Files Categorized)
20. Appendix B — Key Concepts Glossary

THE BIG PICTURE — What Kind of Game Is This?

Before reading any code, you need a mental picture of what the game does and how it's organized.

What the player does: The player controls a character (Dio, Ringo, MoonKnight, Diavolo, or others) running on a scrolling road. Obstacles — cars (Roadrollers) and trains — come from the right. The player must dodge them. Each obstacle that passes scores a point. The player also has special abilities (based on JoJo's Bizarre Adventure) that can be used to destroy obstacles, freeze time, rewind time, and more. The game ends when a hit occurs.

What makes this game complex (and impressive):

- A full **Game State Machine** — Menu, Playing, Paused, Game Over are all separate "scenes."
- A complete **Ability System** — 8 different abilities, each doing something unique.
- A **Time Rewind System** — the game can literally play itself backwards, restoring every actor's past position.
- A **Shop System** — the player earns money and unlocks abilities.
- A **Save System** — playtime, scores, and stats persist across sessions.
- A **Character System** — multiple playable characters, each with their own animations, sounds, and abilities.
- A **Cinematic System** — Boss intro banners that animate across the screen.

HOW GREENFOOT WORKS (The Foundation)

Greenfoot is a Java teaching framework for making games. Before understanding your code, you must understand how Greenfoot itself works, because your entire game is built on top of it.

The World and the Game Loop

Greenfoot has a class called `World`. You extend it with your own class (`MyWorld`). Greenfoot calls your world's `act()` method approximately **60 times per second**. This is the heartbeat of the game — every 1/60th of a second, `act()` fires, and everything in the game updates.

```
Every 1/60 second:
  MyWorld.act() is called
    → gsm.update() is called
      → current state's update() is called
        → all actors update
```

Actors

Everything you see on screen — the player, the cars, the road, the UI text — is an `Actor`. Actors have a position (x, y) in the world, and an image. Each actor also has its own `act()` method that Greenfoot calls every frame. But importantly: **in your game, actors mostly check what state they're in before doing anything**, so the state machine controls behavior.

The Paint Order

Greenfoot draws actors in layers. The `setPaintOrder()` call in `MyWorld()` defines this. The **first** class listed is drawn **on top**. The **last** class listed is drawn at the **bottom** (background). This is why the road scrolls at the bottom and the score text appears on top of everything.

CHAPTER 1 — The Entry Point: `MyWorld.java`

File: `MyWorld.java`

`MyWorld` is the very first class Greenfoot instantiates when the project runs. Everything in the game starts here. It has one field, one constructor, and one method that matters.

The Constructor — What Happens When the Game Starts

```java
public MyWorld() {
    super(GameConfig.WORLD_WIDTH, GameConfig.WORLD_HEIGHT, 1, false);
    SaveManager.load();
    gsm = new GameStateManager(this);
    // ... setPaintOrder ...
    gsm.pushState(new MenuState());
    AudioManager.init();
    ShopManager.init();
}
```

This runs **once**, in this exact order:

1. `super(...)` — Creates the Greenfoot world at the configured size. The `false` at the end means actors are **not** clipped to the boundary (obstacles can exist off-screen to the right before sliding in).
2. `SaveManager.load()` — Reads the save file from disk. High scores, money, playtime, and usernames are all loaded into memory.
3. `new GameStateManager(this)` — Creates the brain of the game. The `this` gives the GSM a reference to the world so it can add/remove actors.

4. `setPaintOrder(...)` — Defines the layering order for all actor types.
5. `gsm.pushState(new MenuState())` — The very first "scene" is pushed. The game opens in the menu.
6. `AudioManager.init()` — Pre-loads all sound files into memory. This prevents lag on first play.
7. `ShopManager.init()` — Sets up which abilities the player has purchased.

The `act()` Method — The Heartbeat

```java
public void act() {
    gsm.update();
}
```

That's it. Everything in the game flows from this one line, 60 times per second. `MyWorld` doesn't do any game logic itself — it delegates entirely to the GameStateManager.

Key Helper Methods

- `getGSM()` — Returns the GameStateManager. Many actors call this to check what state they're in.
- `isRewinding()` — Returns `true` during a time rewind. Actors check this to freeze their movement while the rewind system restores their past positions.
- `getBottomBound()` — Returns how far down actors can move. During the Ability Demo, this is restricted to the top half of the screen.

The Username / Cheat Code

In the constructor, if no username is saved, `requestUsername()` is called. This opens a Java dialog box. If the user types `"mrchan"`, they get 50,000 coins — that's the developer cheat code built into the game.

CHAPTER 2 — The Brain: The Game State Manager

File: `GameStateManager.java`

The Game State Manager (GSM) is the most architecturally important class in your entire project. Without understanding it, nothing else makes sense.

What Problem Does It Solve?

A game has many "scenes" — the main menu, the actual gameplay, the game-over screen, the pause menu, and so on. Naively, you might think about managing these with a bunch of `if/else` statements. The GSM solves this much more cleanly using a **stack**.

Think of the GSM like a stack of plates:

- The **top plate** is always the active scene.
- Pushing a new plate = entering a new scene.
- Popping the top plate = returning to the previous scene.

The Stack in Practice

```
EXAMPLE: Player opens pause menu during gameplay

BEFORE:            AFTER push:
[  PlayingState ]  [  PausedState  ]   ← active (top)
                   [  PlayingState ]   ← paused below
```

When the player unpauses, `PausedState` is **popped**. `PlayingState` is back on top and resumes. `PlayingState` was never destroyed — it was just waiting below.

The Three Operations

`pushState(GameState s)`:

- Takes the new state's `enter()` method — "set up this scene."
- Places it on top of the stack.
- The new state is now active.

`popState()`:

- Calls `exit()` on the current top state — "clean up this scene."
- Removes it from the stack.
- The previous state is now on top again.

`changeState(GameState s)`:

- Calls `exit()` on the current state.
- Removes it.
- Calls `enter()` on the new state.

- Places the new state on the stack.
- This is a **full replacement**, not a pause. Used when going from PlayingState → GameOverState.

`update()` — Called Every Frame

```java
public void update() {
    if (!stack.isEmpty()) {
        stack.peek().update(world);
    }
}
```

`stack.peek()` just looks at the top without removing it. So every frame, only the **top state's** `update()` is called.

`isState(Class)` — A Common Check

Many actors call `world.getGSM().isState(PlayingState.class)` to check if gameplay is currently active. This is how actors know whether to update or freeze.

CHAPTER 3 — The States: All the "Scenes" of the Game

Each of the following classes implements the `GameState` interface, which requires three methods: `enter()`, `update()`, and `exit()`.

`MenuState.java` — The Main Menu

What it does: Displays the background, title art, and navigation buttons (Play, Shop, Dossier, etc.).

`enter()`: Creates and adds UI buttons and background actors to the world. Starts the menu background music.

`update()`: Listens for keyboard input or button clicks. When the player presses a button to start the game, it calls `gsm.pushState(new CharacterSelectState())`.

`exit()`: Removes all UI elements it created. Stops menu music.

`CharacterSelectState.java` — Character Selection

What it does: Displays character portraits and lets the player choose who to play as.

`update()`: When a character is selected, it sets `GameConfig.ACTIVE_CHARACTER` (a global variable) to the chosen character, then calls `gsm.changeState(new PlayingState())` to start the game.

`PlayingState.java` — The Main Gameplay

This is the most important state. When it's active, the game is being played.

`enter()`:

1. Removes all existing actors from the world (a clean slate).
2. Resets the score to zero.
3. Starts the character's background music.
4. Creates two `ScrollingRoad` actors (the moving road background).
5. Creates the `SpawnManager` (handles spawning obstacles over time).
6. Creates the score display text.
7. Spawns the player character (a `Dio` object if Dio is selected, otherwise a `GenericPlayer`).
8. Creates the `UI_RewindBar` if the player has Mandom.
9. Creates `UI_AbilityIcon` objects for all visible abilities.

`update()`: Every frame, it either:

- **Records** the current world state into the rewind history buffer (normal play), OR
- **Rewinds** one frame of history (during a rewind).
- Updates the spawn manager.
- Updates the score display text.

`exit()`: Saves playtime and money earned. Removes all actors.

`PausedState.java` — The Pause Menu

When Dio uses "The World" (time stop), this state is pushed on top of `PlayingState`. Actors check `isState(PausedState.class)` and freeze.

`GameOverState.java` — The Death Screen

Pushed when the player's death timer expires.

`enter()`:

1. Plays the death music.
2. Reads the final score and all-time high from `SaveManager`.

3. Displays the score, the all-time record, a randomly chosen obituary line, and playtime stats.

`update()`: Waits for the player to press Enter, then calls `gsm.changeState(new MenuState())`.

`AbilityDisplayState.java` — The Tutorial Demo

A special interactive tutorial mode that shows the player how each ability works. It restricts movement to the top portion of the screen and uses a scripted `DemoStage` timeline for spawning obstacles.

`LegacyState.java` — The "Stand Dossier"

An overlay state showing the player's lifetime statistics — all-time best score, total playtime, favorite character, most-used ability, and a "personality verdict" based on their play style. Accessible from the main menu.

`ShopState.java` — The Ability Shop

Allows the player to spend earned coins on abilities. Uses `ShopManager` to read and write what's been purchased.

CHAPTER 4 — The Player: `Player`, `GenericPlayer`, and `Dio`

There are three classes that make up the player hierarchy. Understanding their relationship is essential.

The Hierarchy

```
Player (interface — the contract)
  └── GenericPlayer extends Actor implements Player (the real player)
        └── Dio extends GenericPlayer (a special subclass for Dio)
```

`Player.java` — The Interface (The Contract)

An **interface** in Java is a list of methods that any implementing class **must** provide. `Player` defines:

- `die()` — Called when the player is hit.
- `isDead()` — Returns true if dead.
- `checkCustomHitbox(Actor, double)` — Circular collision check.

This interface exists so that `Roadroller` and `Train` can call `player.die()` without needing to know whether it's a `GenericPlayer` or a `Dio`. Either way, the method exists.

`GenericPlayer.java` — The Real Player

This is where almost all player logic lives. It's a large class. Here's what it manages:

Fields (the data it stores):

- `config` — A `CharacterConfig` object with all the character's data (speed, animations, sounds).
- `animations` — A `HashMap<String, Animator>` mapping animation names ("Dash", "Lose", "Wry") to their `Animator` objects.
- `currentAnimator` / `currentAnimName` — Which animation is currently playing.
- `abilities` — A `List<Ability>` of all equipped abilities.
- `isDead` — Whether the player is currently dead.
- `iFrameTimer` — A `GameTimer` counting down invincibility frames after getting hit.
- `deathTimer` — A `GameTimer` counting down the 4 seconds before the Game Over screen.

Constructor — `GenericPlayer(CharacterConfig config)`:

Two important things happen here:

1. **Loading animations:** For each animation name in `config.animNames`, a new `Animator` is created and stored in the `animations` map.
2. **Loading abilities via Reflection:** This is the most advanced piece of code in the class. For each ability class name (e.g., `"Ability_StandPunch"`) in `config.abilityClassNames`, it uses Java Reflection (`Class.forName(className).getDeclaredConstructor().newInstance()`) to create an instance. This allows the character's ability list to be defined in data (CharacterConfig) rather than hardcoded in code.

`act()` — The Player's Per-Frame Logic:

Greenfoot calls this every frame. But the player delegates its logic:

```java
```

```
public void act() {
    // Get the current game state
    // If it's an active gameplay state (PlayingState / AbilityDisplayState):
    //   Call movementLogic() and animationLogic()
    // If it's a PausedState:
    //   Call onPauseUpdate() (for special pause behavior like Dio's Wry pose)
}
```

`movementLogic()` — Per-Frame Player Logic:

Called every gameplay frame. The order is important:

1. Update the i-frame timer.
2. Update every ability's internal timers.
3. Check if an ability's key is pressed → activate it.
4. If dead: handle the death sequence (shake, then transition to GameOver). Check for the King Crimson easter egg.
5. If alive and not rewinding: call `handleStandardMovement()`.

`handleStandardMovement()`: Reads UP/DOWN arrow keys and moves the player. Clamps the position to stay within the world bounds. If `Ability_MadeInHeaven` is active, speed is doubled.

`die()`: Called by obstacles on collision. Several conditions block death:

- I-frames are active (just got hit, or just teleported).
- Player is already dead.
- `Ability_StandPunch` is active (you're invincible while punching).
- A `shouldHidePlayer()` ability is active (underground = invincible).

If death proceeds:

- Sets `isDead = true`.
- Cancels all abilities.
- Switches animation to "Lose".
- Plays the character's death voice.
- Starts the 4-second death timer.

`animationLogic()`: Sets the actor's image to the current frame of the current animation. Handles:

- Invisibility when underground (Sticky Fingers).
- I-frame blinking (alternates transparency every 4 frames).
- Debug mode: draws the hitbox circle over the sprite.

`Dio.java` — The Dio Subclass

`Dio` extends `GenericPlayer`. It has only **one meaningful addition**: the `onPauseUpdate()` method.

```java
protected void onPauseUpdate(MyWorld world) {
    if (!bannerSpawned) {
        world.addObject(new Banner(CharacterConfig.DIO.bossConfig), ...);
        bannerSpawned = true;
        setAnimation("Wry");
    }
    movementLogic();
    animationLogic();
}
```

When Dio uses The World (which pushes `PausedState`), `onPauseUpdate()` is called instead of normal logic. It spawns the boss intro banner once, locks Dio in the "Wry" pose, and continues updating abilities and animation.

CHAPTER 5 — The Obstacles: `Obstacles`, `Roadroller`, and `Train`

The Hierarchy

```
Actor (Greenfoot base)
  └── Obstacles (abstract base with shared logic)
        ├── Roadroller (the standard car)
        └── Train (the fast charge enemy)
```

`Obstacles.java` — The Abstract Base

An **abstract class** is like an interface but it can also contain real code. Classes that extend `Obstacles` inherit its `act()` method and its `anim` field, but they **must** implement three abstract methods themselves.

`act()` — The Shared Per-Frame Logic:

```java
public void act() {
    // Guard: only run during active gameplay and not while rewinding
    if (!isActiveGameplay()) return;
    animUpdate();    // advance animation frames
    movementLogic(); // move the obstacle (implemented by subclasses)
    collisionLogic();// check for player collision (implemented by subclasses)
    checkRemove();   // remove if off-screen (implemented by subclasses)
}
```

The Three Abstract Methods (subclasses must implement these):

- `movementLogic()` — How does this obstacle move?
- `collisionLogic()` — What happens when it touches the player?
- `checkRemove()` — When should it remove itself?

`Roadroller.java` — The Standard Car

The most common obstacle. Spawned continuously by `SpawnManager`.

Constructors (three of them):

- `Roadroller()` — Default speed from `GameConfig.ROADROLLER_SPEED`, awards 1 point.
- `Roadroller(int speed)` — Custom speed, still awards 1 point.
- `Roadroller(int speed, int scoreToAdd)` — Full custom. Used for scoreless obstacles.

Note how the constructors chain each other: `Roadroller(int speed)` calls `this(speed, 1)`, which calls the full constructor, which calls `this()` (the default). This is called **constructor chaining** and avoids repeating the sprite-loading code.

`movementLogic()`: Simply `move(-speed)` — moves left each frame.

`collisionLogic()`: Calls `player.checkCustomHitbox(this, 0.8)` — a circular distance check with a slightly forgiving 80% radius. If hit, calls `player.die()`.

`checkRemove()`: If `getX() <= 0`, the car has passed the left edge. Award the score (if the player is alive and not hidden), then `removeObject(this)`. Note: **`removeObject(this)` must always be the absolute last line.** After removal, `getWorld()` returns `null` — calling any method after that would crash.

`Train.java` — The Fast Charge Enemy

The Train is the dangerous obstacle. Unlike the Roadroller, it waits for a moment before charging

across the full screen at high speed.

Sequence of events when a Train is spawned:

1. `SpawnManager` calls `spawnTrain()`.
2. A `PathWarning` (red lane flash) appears at the train's lane.
3. An `Exclaimation` mark (`!`) appears at the right edge.
4. The `Train` actor is added, but **starts invisible and stationary** for ~65 frames.
5. After 65 frames, it charges left at high speed.
6. The `PathWarning` disappears 5 frames before the charge — the brief "oh no" moment.

Score: Awards 5 points per train dodged (vs 1 for a Roadroller), because trains are harder to dodge.

CHAPTER 6 — The Ability System: The `Ability` Interface

File: `Ability.java`

This is the second-most important interface in the project. It defines the contract that every ability must follow.

The Interface Methods

```java
public interface Ability {
    void activate(Player p, MyWorld world);  // Called when the key is pressed
    void update(Player p, MyWorld world);   // Called every frame (for timers, effec
    void cancel();                          // Called when player dies, resets abili
    boolean isActive();                     // Is the ability's effect currently run
    boolean isCooldownActive();             // Is the cooldown timer running?
    double getActivePercent();              // For the ability icon fill (0.0 to 1.0
    double getCooldownPercent();            // For the cooldown fill (0.0 to 1.0)
    String getKeybind();                    // Which key triggers this (e.g., "e")
    String getDisplayLabel();              // The letter shown on the ability icon
    Object captureState();                 // For the time rewind system
    void restoreState(Object state);       // For the time rewind system
}
```

There are also **default methods** in the interface — these have a built-in implementation that subclasses can override if needed:

- `shouldHidePlayer()` — Returns `false` by default. Only `Ability_StickyFingers` overrides this to `true` (the player is underground).
- `shouldShowIcon()` — Returns `true` by default. `Ability_Mandom` overrides to `false` because the rewind bar already shows its state.
- `getMovementMultiplier()` — Returns `1.0` by default. `Ability_MadeInHeaven` overrides to `2.0`.

How Abilities Are Called

Inside `GenericPlayer.movementLogic()`, every frame:

```java
for (Ability a : abilities) {
    a.update(this, world);  // always update (timers tick)

    if (Greenfoot.isKeyDown(a.getKeybind())) {
        a.activate(this, world);  // activate when key is down
    }
}
```

The key insight: `update()` always runs (it ticks down timers), but `activate()` only runs when the key is held down. Inside `activate()`, most abilities check if they're already active or on cooldown, and return early if so.

`captureState()` and `restoreState()` — The Time Machine Hook

Every ability must save its entire state (timer values, active flags, etc.) into an object, and be able to restore from that object. This is how the time rewind system works for abilities. More on this in Chapter 8.

CHAPTER 7 — All the Abilities Explained One by One

`Ability_TheWorld` — Za Warudo (Key: W)

Character: Dio

What it does: Toggles time stop. When active, `TIME_STOPPED` (a `public static boolean`) is set to `true`. The `PausedState` is pushed on top of `PlayingState`. All obstacles freeze. Dio strikes his "Wry" pose.

No cooldown: This ability is a simple toggle. No timer. It stays active until the player presses W again.

`Ability_StandPunch` — Ora Ora Ora (Key: E)

Character: Dio

What it does: Spawns `TheWorldStand` beside Dio. The Stand follows Dio and destroys any obstacle it touches, awarding 2 bonus points per hit. Dio is invincible while this is active.

How `TheWorldStand` works:

- Each frame, it checks `ability.isActive()`. If the ability ended, it removes itself immediately.
- It finds the `GenericPlayer` in the world, and positions itself 80 pixels to the right.
- It checks `getIntersectingObjects(Obstacles.class)` and removes any obstacle it touches.

Duration + cooldown: Active for a set time (`STAND_PUNCH_DURATION` seconds), then enters a cooldown period.

`Ability_KingCrimson` — Epitaph / Time Erase (Key: Q)

Character: Diavolo

What it does: Skips forward in time. All obstacles in the world are instantly fast-forwarded — the player passes them without collision. A red overlay fills the screen. A cinematic King Crimson overlay plays.

`ERASING` static boolean: Like `TIME_STOPPED`, this static flag tells other actors (especially `isGameFrozen()`) that the erase is in progress. Obstacles skip movement. Collision is suppressed.

The King Crimson Easter Egg: When Diavolo is dead and the player presses the RIGHT arrow key, a secret cinematic plays. A fake game over appears, then Diavolo "erases" his own death. This is a fun homage to the character's ability.

`Ability_Mandom` — Time Rewind (Key: R)

Character: Ringo Roadagain

What it does: Triggers the `Time_RewindManager` to play back the last several seconds of gameplay in reverse. The world literally un-happens. See Chapter 8 for the full explanation of how this works.

No traditional duration/cooldown: The ability's "cooldown" is just the rewind bar emptying. Once it's empty, you can't rewind anymore.

`Ability_StickyFingers` — Underground Portal (Key: Q)

Character: Giorno / Bruno style ability

What it does: Warps the player underground. The player becomes **invisible and invincible**. They teleport from one vertical edge to the other (top → bottom or bottom → top). Two `FX_Portal` (zipper) effects flash at the screen edges. An `FX_ZipperGround` draws a zipper on the road while underground.

`shouldHidePlayer()` returns `true`: While this ability is active, the player sprite's transparency is set to 0 (invisible). No score is awarded while underground (since you're not really dodging — you're cheating your way past obstacles).

`Ability_MadeInHeaven` — Ultra Speed (Key: Q)

Character: Pucci-style

What it does: Doubles the player's movement speed. Also dramatically increases the game simulation speed (`Greenfoot.setSpeed()`), making obstacles come in faster. The road scrolls faster. The entire world accelerates.

`getMovementMultiplier()` returns `2.0`: `GenericPlayer.handleStandardMovement()` checks for this and doubles the speed.

`Ability_DarkSpell01` — Void Sphere / Annihilation Beam (Key: V)

Character: Moon Knight

What it does (normal mode): Creates a dark energy sphere around the player. All obstacles within a radius are instantly destroyed. Duration: a few seconds.

What it does (resonance / combo mode): If `Ability_DarkSpell02` has frozen a lane (`CURRENT_FROZEN_LANE_Y != -1`), then pressing V instead fires an **Annihilation Beam** across the frozen lane. Every obstacle in that lane is destroyed in a dramatic beam effect. The combo mode has a shorter cooldown as a reward.

`Ability_DarkSpell02` — Lane Freeze (Key: C)

Character: Moon Knight

What it does: Freezes all obstacles in a specific horizontal lane. A dark blue `FX_DarkLane` effect highlights the lane. Frozen obstacles stop moving. The `CURRENT_FROZEN_LANE_Y` static variable stores which lane's Y coordinate is frozen, so `Ability_DarkSpell01` can read it for the resonance combo.

CHAPTER 8 — The Time Machine: Rewinding Time

This is the most technically sophisticated system in the project. It consists of three classes working together.

The Three Pieces

Class	Role
`Time_RewindManager`	The "DVD player" — manages recording and playback
`Time_ActorMemento`	One "frame" of one actor's state (position + custom data)
`Time_Snapshottable`	Interface for actors that want to participate in rewinding

The Core Idea — The Memento Pattern

Every frame during normal gameplay, the `Time_RewindManager` takes a **snapshot** of every actor in the world. A snapshot stores:

- Which actor it is (a direct reference).
- The actor's X and Y position at that moment.
- Any extra data the actor wants to save (speed, timer state, etc.).

These snapshots are stored in a **deque** (double-ended queue). Think of it as a VHS tape. Every frame records a new moment onto the tape. The tape has a maximum length (`MAX_REWIND_TIME`). When full, the oldest frame is discarded.

Recording (Normal Gameplay)

```java
// Called every frame by PlayingState.update()
rewindManager.record(world, spawnManager);
```

Inside `record()`:

1. For every actor in the world that implements `Time_Snapshottable`, call `actor.captureState()`.
2. Bundle all these mementos into one `WorldSnapshot`.
3. Add the snapshot to the **front** of the deque.
4. If the deque is too long, remove the **back** (oldest).

Rewinding (Mandom Activated)

```java
// Called every frame during rewind by PlayingState.update()
rewindManager.rewindStep(world, spawnManager, rewindSpeed);
```

Inside `rewindStep()`:

1. Take the most recent snapshot from the deque.
2. For each actor memento in that snapshot:
   - If the actor is still in the world: restore its position and call `actor.restoreState()`.
   - If the actor is NOT in the world (it was destroyed): add it back and restore its state.
3. Remove actors from the world that didn't exist at that past moment.
4. Restore the score from the snapshot.
5. Also restore the `SpawnManager`'s internal timing state (so spawning resumes correctly after rewind).

`Time_Snapshottable` — The Interface

Any actor that wants the rewind system to track it must implement this interface:

```java
```

```
public interface Time_Snapshottable {
    Time_ActorMemento captureState();
    void restoreState(Time_ActorMemento m);
}
```

Example — Roadroller's implementation:

```java
public Time_ActorMemento captureState() {
    return new Time_ActorMemento(this, getX(), getY(),
        new RoadrollerData(speed, scoreToAdd));
}

public void restoreState(Time_ActorMemento m) {
    RoadrollerData d = (RoadrollerData) m.customData;
    this.speed = d.speed;
    this.scoreToAdd = d.scoreToAdd;
}
```

The `customData` field in `Time_ActorMemento` is typed as `Object` — it can hold anything. Each actor knows what it stored and casts it back when restoring.

`captureState()` and `restoreState()` on Abilities

Abilities also have these methods (defined in the `Ability` interface, but these save timers and flags, not positions). Inside `GenericPlayer.captureState()`, all ability states are captured. This is how ability timers rewind correctly — if you had 2 seconds left on a cooldown and you rewind 3 seconds, you'll have 5 seconds left afterward.

Why Some Actors Don't Need It

`FX_Portal`, `FX_DimOverlay`, and many other visual effects do **not** implement `Time_Snapshottable`. They are purely cosmetic and short-lived. The rewind system ignores them, and they may appear or disappear oddly during rewind — but this is acceptable because they're just eye candy.

CHAPTER 9 — The Manager Classes (The Specialists)

These are all `static` utility classes — no instances. You call their methods as `ManagerName.methodName()`.

`GameConfig.java` — The Configuration Hub

This is the most-referenced file in the entire project. It stores every constant and configuration value. There are almost no "magic numbers" scattered throughout the codebase — they're all defined here.

Key things in `GameConfig`:

- `WORLD_WIDTH`, `WORLD_HEIGHT` — Size of the game world.
- `SCALE` — A multiplier for all sizes/positions. Changing this scales the entire game.
- `s(int value)` — A helper method: `return (int)(value * SCALE)`. Every dimension in the game goes through `GameConfig.s()`.
- `ACTIVE_CHARACTER` — Which `CharacterConfig` is currently selected.
- `LANES[]` — The Y positions of each road lane.
- `ROADROLLER_SPEED`, `ROADROLLER_MIN_RATE`, etc. — Difficulty constants.
- `DEBUG_MODE` — A boolean. When `true`, hitbox circles are drawn over player sprites.

`AudioManager.java` — Sound Control

All sounds go through `AudioManager`. It has several key features:

- **`init()`:** Pre-loads all sounds into `GreenfootSound` objects. This prevents the first-play lag spike.
- **`play(key)`:** Plays a one-shot sound effect.
- **`playLoop(key)`:** Starts a looping sound (background music).
- **`stop(key)`:** Stops a sound.
- **`playPool(key)`:** Picks a random sound from a named group and plays it. Used for death voices ("Dio says one of 5 random things when he dies") and battle cries.
- **`stopAllAbilities()`:** Stops all currently-playing ability sounds (called on player death).
- **`setAllSoundsPaused(true/false)`:** Mutes/unmutes everything — used during rewind (rewind should be silent).

`SaveManager.java` — Persistent Data

Reads and writes a plain-text `user_stats.txt` file in the project directory.

What it stores:

- `username` — The player's entered name.
- `money` — Total coins earned.
- `all_time_high` — All-time best score.

- `total_playtime` — Total seconds played.
- `time_Dio`, `time_Ringo`, etc. — Time played per character.
- `use_Ability_StandPunch`, etc. — How many times each ability was used.

The data format: Simple `key=value` lines in the text file.

API: `getInt("key")`, `setInt("key", value)`, `addInt("key", amount)`, `getUsername()`, `setUsername(name)`, `save()`, `load()`.

`ScoreManager.java` — Current Session Score

A fully static class. Methods:

- `addScore(int)` — Called by Roadroller (+1) and Train (+5) when obstacles pass.
- `setScore(int)` — Called only by the rewind system to restore past score values.
- `reset()` — Called at the start of each new game.
- `updateHighScore()` — Saves to `SaveManager` if a new record was set. Ignores the "cheat" character `omnipotent_DIO`.
- `getScore()` — Returns the current score.

`SpawnManager.java` — Obstacle Spawning

Handles the entire difficulty progression system.

How it works: Every frame, it checks internal counters:

- `roadrollerTimer` counts up. When it reaches `roadrollerRate`, a new Roadroller is spawned and the timer resets.
- `trainTimer` counts similarly for Trains.
- As the game progresses, `roadrollerRate` decreases (faster spawning) and obstacle speed increases.

Difficulty scaling: There are configurable constants in `GameConfig` for minimum spawn rate, maximum speed, and how fast the difficulty ramps up.

Random lanes: Uses `GameRNG` to pick which lane to spawn in. `GameRNG` uses a seeded random number generator — same seed = same obstacle pattern. This is why the time rewind can correctly regenerate the same sequence.

`ShopManager.java` — The Ability Shop

- **`init()`:** Reads the `SaveManager` to build a set of unlocked abilities.

- `isUnlocked(String key)`: Returns `true` if the player has purchased the given item.
- `purchase(String key, int cost)`: Deducts money and marks the item as purchased. Saves to disk.

`ObituaryManager.java` — Death Quotes

Reads `.txt` files from the `obituaries/` folder. Each character has their own file (e.g., `obituaries_Dio.txt`). Each line is one possible death quote. A random one is displayed on the Game Over screen. Caches the loaded quotes in a `HashMap` so the file is only read once.

`GameRNG.java` — The Random Number Generator

A seeded random number generator. `randomize()` picks a new random seed. Because it's seeded, the rewind system can replay the same "random" obstacle sequence exactly — the same seed produces the same sequence every time.

CHAPTER 10 — The Animation System: `Animator`

File: `Animator.java`

Every animated sprite in the game — player dashing, obstacle rolling, the "!" warning mark — goes through this class.

What It Does

An `Animator` loads a sequence of numbered PNG files from a folder and cycles through them at a fixed rate. For example:

```
images/Dio/Dash/Dash_000.png
images/Dio/Dash/Dash_001.png
images/Dio/Dash/Dash_002.png
...
```

Three Constructors (Most to Least Detailed)

1. **Full constructor:** `(baseFolder, folderName, prefix, frameCount, speed, scaleFactor)` — Maximum control. Rarely used directly.
2. **Medium constructor:** `(baseFolder, folderName, speed, scaleFactor)` — Auto-counts frames by scanning the folder.
3. **Short constructor:** `(baseFolder, folderName, scaleFactor)` — Auto-counts frames, uses the default speed (6 ticks/frame = ~10 fps).

Auto Frame Counting

The `countFrames()` method uses Java's `File` class to look inside the folder and count how many `.png` files exist. **This means you can add more frames to an animation folder without changing any code.** The Animator will find and use them automatically.

`getCurrentFrame()`

This is called every game tick by whichever actor uses this Animator. It:

1. Increments an internal `timer`.
2. When `timer >= speed`: advances `currentFrame` to the next index, resets `timer`.
3. Returns the pre-loaded `GreenfootImage` at `currentFrame`.
4. If `loop = true`, wraps from the last frame back to frame 0. If `loop = false`, stays on the last frame (used for death animations).

How `GenericPlayer` Uses It

`GenericPlayer` stores a `HashMap<String, Animator>` — one `Animator` per animation name. When `setAnimation("Dash")` is called, it switches which `Animator` is current. When `animationLogic()` runs, it calls `currentAnimator.getCurrentFrame()` and sets that as the actor's image.

CHAPTER 11 — The Characters: `CharacterConfig` and `BossConfig`

`CharacterConfig.java` — The Data for Each Character

`CharacterConfig` is a Java `enum`. An enum is a fixed set of named constants. Each constant is a full `CharacterConfig` object with all the data for that character.

What each `CharacterConfig` stores:

- `folderName` — The folder in `images/` where the character's sprites live (e.g., `"Dio"`).
- `animNames` — A list of animation names this character has (e.g., `["Dash", "Wry", "Lose"]`).
- `defaultAnim` — Which animation plays first (usually `"Dash"`).
- `abilityClassNames` — A list of class names (e.g., `["Ability_TheWorld", "Ability_StandPunch"]`). These are loaded via Reflection.
- `moveSpeed` — How fast this character moves vertically.
- `scale` — Size multiplier for this character's sprites.

- `bgmKey` — The `AudioManager` key for this character's background music.
- `deathSoundKey` — The key for the pool of death voice sounds.
- `displayName` — The human-readable name shown in UI.
- `roadImage` — Which road/obstacle skin to use.
- `bossConfig` — A `BossConfig` for the boss intro banner (or `null` if this character has none).

Example: `CharacterConfig.DIO` is a constant that holds all of Dio's data. When Dio is selected, `GameConfig.ACTIVE_CHARACTER = CharacterConfig.DIO` is set, and everything else reads from that.

`BossConfig.java` — The Boss Intro Banner Data

Also an enum. Each constant defines the visual data for one character's boss intro banner:

- `bgColor` — The background color of the banner strip.
- `soundsKey` — AudioManager pool key for the battle cry voice.
- `overlays` — An array of `Banner.SpriteOverlay` objects, each specifying a sprite image and its position offset on the banner canvas.

Currently defined: `BossConfig.DIO` and `BossConfig.Ringo`.

CHAPTER 12 — The Configuration Hub: `GameConfig`

Everything important about the game's settings lives here. Rather than listing every constant, here are the most important categories:

Screen/World:

- `WORLD_WIDTH`, `WORLD_HEIGHT` — The pixel dimensions of the game.
- `SCALE` — One number controls the entire game's scale.
- `s(int)` — Scale converter. Every position/size goes through this.

Road/Lanes:

- `LANES[]` — Array of Y coordinates for each lane. Obstacles are spawned at these Y values.
- `LANE_HEIGHT` — How tall each lane is.

Difficulty:

- `ROADROLLER_SPEED`, `ROADROLLER_MIN_RATE`, `ROADROLLER_MAX_RATE` — Speed and spawn frequency bounds.
- `DIFFICULTY_RAMP_RATE` — How quickly the game gets harder over time.

Characters:

- `ACTIVE_CHARACTER` — Set when a character is selected. Read by nearly everything.

Abilities:

- Constants for each ability's duration, cooldown, radius, etc. (e.g., `DS01_RADIUS`, `DS01_COOLDOWN`).

Debug:

- `DEBUG_MODE` — Shows hitbox circles when `true`.

Road skins:

- `getActiveRoad()`, `getObstacleImage()`, `getPathWarningColor()` — Return different asset paths based on the active character's `roadImage` value.

CHAPTER 13 — The User Interface (UI Classes)

These classes are all actors placed in the world to display information. They don't interact with gameplay — they only read data and display it.

`UIText.java`

The most-used UI class. Creates a text label as a `GreenfootImage`. Features:

- Configurable font size and color.
- **Text wrapping:** If a `maxWidth` is provided, text is automatically broken into multiple lines using a `BufferedImage` canvas to measure text width.
- `setText(String)` — Updates the text and re-renders the image.

`UI_AbilityIcon.java`

Displays a circular icon for one ability. Shows:

- A base circle (always visible).

- A filled arc showing the **active duration** remaining (shrinks as the ability wears off).
- A filled arc showing the **cooldown** (shrinks as the cooldown expires).
- A letter label (the keybind, e.g., "E").

Updated every frame by checking `ability.getActivePercent()` and `ability.getCooldownPercent()`.

`UI_RewindBar.java`

A horizontal bar showing how much rewind time is available. Reads directly from the `Time_RewindManager` to know how full the buffer is.

`UI_Panel.java`

A simple solid colored rectangle. Used as a background panel for other UI elements. If the color has `blue == 30`, it adds a thin white top border.

`UI_MenuBackground.java`

Loads an image and scales it to fill the entire world. Used as the background in the menu.

`UI_HighlightBox.java`

A highlighted box drawn around UI elements for visual emphasis.

`UI_Preview.java`

Shows an animated character preview in the character selection screen. Uses an `Animator` to cycle through the selected character's "Dash" animation frames.

`Banner.java`

The cinematic boss intro banner. A self-animating actor with three phases:

1. **Slide in** from the right, easing toward the center.
2. **Hold** for a moment, then accelerate rightward while squeezing vertically.
3. **Slide out** at increasing speed with fading alpha.

The banner reads all its visual data from a `BossConfig` — the colors, the sprite overlays, and the sound. Renders itself each frame onto a large transparent canvas.

CHAPTER 14 — The Visual Effects (FX Classes)

These actors are purely visual. Most have a short lifetime, then remove themselves. Most do **not** implement `Time_Snapshottable` because they're cosmetic.

Class	What it looks like	How long
`FX_DimOverlay`	Semi-transparent black overlay	Permanent (until removed)
`FX_DarkLane`	Blue transparent rectangle over one lane	30 frames, fades out
`FX_TimeStopOverlay`	Color-altering effect during time stop	Permanent until removed
`FX_RewindOverlay`	Blue scanline effect during rewind	Permanent while rewinding
`FX_Portal`	Purple/gold zipper strip at screen edge	~18 frames, fades
`FX_ZipperGround`	Zipper teeth drawn on the road	Follows player while underground
`FX_KingCrimsonOverlay`	Red overlay during KC activation	Until KC ends
`FX_ErasureSnap`	Flash effect during KC erase	Short burst
`FX_ShatteredGlass`	Glass shattering effect	Animated, then removed
`FX_DarkAnnihilationBeam`	The beam that fires during DS01 combo	Flies across screen
`FX_Afterimage`	Ghost trail behind the player	~18 frames, fades
`FX_FakeGameOver`	The fake "Game Over" screen in KC easter egg	Until KC sequence finishes

---

CHAPTER 15 — The Interfaces and Contracts

Interfaces are Java's way of creating a shared language between classes that don't know about each other. Your project has five key interfaces.

`GameState` — The Three-Method Contract

```java
void enter(MyWorld world);
void update(MyWorld world);
void exit(MyWorld world);
```

Implemented by: `MenuState`, `CharacterSelectState`, `PlayingState`, `PausedState`, `GameOverState`, `AbilityDisplayState`, `LegacyState`, `ShopState`.

`Player` — The Player Contract

```java
void die();
boolean isDead();
boolean checkCustomHitbox(Actor attacker, double padding);
```

Implemented by: `GenericPlayer` (and `Dio` by inheritance).

`Ability` — The Ability Contract

All the methods described in Chapter 6.

Implemented by: `Ability_TheWorld`, `Ability_StandPunch`, `Ability_KingCrimson`, `Ability_Mandom`, `Ability_StickyFingers`, `Ability_MadeInHeaven`, `Ability_DarkSpell01`, `Ability_DarkSpell02`.

`IActiveGameState` — The "Is the Game Running?" Contract

```java
boolean isRewinding();
boolean isGameFrozen();
```

Implemented by `PlayingState`. Used by actors (via `world.getGSM().peekState() instanceof IActiveGameState`) to check whether they should update. The key insight: both `PlayingState` and `AbilityDisplayState` are "active gameplay" — they both implement this interface. `PausedState` and `MenuState` do not.

`Time_Snapshottable` — The Rewind Contract

```java
Time_ActorMemento captureState();
void restoreState(Time_ActorMemento m);
```

Implemented by: `GenericPlayer`, `Roadroller`, `Train`, `Exclaimation`, `PathWarning`, `TheWorldStand`, `ScrollingRoad`.

CHAPTER 16 — The Full Data Flow: One Frame of Gameplay, Step by Step

Now let's trace exactly what happens in one single frame (1/60th of a second) during normal gameplay. This is the "everything connected" view.

```
FRAME START (Greenfoot calls MyWorld.act())
|
├── MyWorld.act()
|     └── gsm.update()
|           └── PlayingState.update(world)     ← top of GSM stack
|                 |
|                 ├── [CHECK: is rewinding?] NO → record current state
|                 |     └── rewindManager.record(world, spawnManager)
|                 |           ├── For each Time_Snapshottable actor:
|                 |           |     actor.captureState() → Time_ActorMemento
|                 |           └── Store WorldSnapshot in deque
|                 |
|                 ├── spawnManager.update(world)
|                 |     ├── Increment timers
|                 |     ├── If roadrollerTimer >= roadrollerRate:
|                 |     |     spawnRoadroller() → new Roadroller added to world
|                 |     └── If trainTimer ready:
|                 |           spawnTrain() → PathWarning + Exclaimation + Train
added
|                 |
|                 └── scoreDisplay.setText("SCORE: " + ScoreManager.getScore())
|
| [Greenfoot now calls act() on every Actor in the world]
|
├── GenericPlayer.act()
|     ├── Get current state → is it IActiveGameState? YES → run logic
|     ├── movementLogic()
|     |     ├── iFrameTimer.update()
|     |     ├── For each Ability:
|     |     |     a.update(player, world)     ← timers tick
|     |     |     if key down: a.activate()   ← ability fires
|     |     └── handleStandardMovement()      ← UP/DOWN arrow movement
|     └── animationLogic()
|           └── setImage(currentAnimator.getCurrentFrame())
|
├── Roadroller.act()   (x however many are in the world)
|     ├── isActiveGameplay()? YES
|     ├── animUpdate() → advance animation frame
|     ├── movementLogic() → move(-speed)
|     ├── collisionLogic()
|     |     └── player.checkCustomHitbox(this, 0.8)
|     |           └── if hit: player.die()
|     └── checkRemove()
```

```
│           └── if getX() <= 0:
│                 ScoreManager.addScore(1)
│                 world.removeObject(this)
│
├── ScrollingRoad.act()
│     └── Scroll left by road speed, wrap when off-screen
│
├── UIText.act()   (score display, etc.)
│     └── Redraw image with current text
│
└── [All other actors do their thing]

FRAME END
```

APPENDIX A — The Complete File Map (All 76 Files Categorized)

Entry Point

- `MyWorld.java` — Greenfoot world. Starts everything.

Architecture

- `GameStateManager.java` — The GSM stack.
- `GameState.java` — Interface for all states.
- `IActiveGameState.java` — Interface for gameplay states.

Game States

- `MenuState.java`
- `CharacterSelectState.java`
- `PlayingState.java`
- `PausedState.java`
- `GameOverState.java`
- `AbilityDisplayState.java`
- `LegacyState.java` (Stand Dossier)
- `ShopState.java`

- `Player.java` — Interface.
- `GenericPlayer.java` — Main player class.
- `Dio.java` — Dio-specific subclass.

Obstacles

- `Obstacles.java` — Abstract base.
- `Roadroller.java`
- `Train.java`

Abilities

- `Ability.java` — Interface.
- `Ability_TheWorld.java`
- `Ability_StandPunch.java`
- `Ability_KingCrimson.java`
- `Ability_Mandom.java`
- `Ability_StickyFingers.java`
- `Ability_MadeInHeaven.java`
- `Ability_DarkSpell01.java`
- `Ability_DarkSpell02.java`

Time / Rewind System

- `Time_RewindManager.java`
- `Time_ActorMemento.java`
- `Time_Snapshottable.java`

Manager / Utility Classes

- `GameConfig.java` ← configuration hub
- `AudioManager.java`
- `SaveManager.java`
- `ScoreManager.java`
- `SpawnManager.java`

- `ShopManager.java`
- `ObituaryManager.java`
- `GameRNG.java`
- `GameTimer.java`

Configuration Data

- `CharacterConfig.java`
- `BossConfig.java`
- `DemoStage.java`

Animation

- `Animator.java`

UI Actors

- `UIText.java`
- `UI_AbilityIcon.java`
- `UI_RewindBar.java`
- `UI_Panel.java`
- `UI_MenuBackground.java`
- `UI_HighlightBox.java`
- `UI_Preview.java`
- `Banner.java`

Visual Effects (FX)

- `FX_DimOverlay.java`
- `FX_DarkLane.java`
- `FX_TimeStopOverlay.java`
- `FX_RewindOverlay.java`
- `FX_Portal.java`
- `FX_ZipperGround.java`
- `FX_KingCrimsonOverlay.java`
- `FX_ErasureSnap.java`

- `FX_ShatteredGlass.java`
- `FX_DarkAnnihilationBeam.java`
- `FX_Afterimage.java`
- `FX_FakeGameOver.java`

Gameplay Actors

- `ScrollingRoad.java`
- `Exclaimation.java`
- `PathWarning.java`
- `TheWorldStand.java`

Misc / Abandoned

- `SadFace.java` — Unused. Safe to delete.

APPENDIX B — Key Concepts Glossary

Abstract class: A class that cannot be instantiated directly. It can have both implemented and unimplemented (abstract) methods. Used by `Obstacles` — you can't create a plain `Obstacles`, only a `Roadroller` or `Train`.

Actor: A Greenfoot object that lives in the world and has a position and image. Has an `act()` method called every frame.

Deque: A "double-ended queue" — a list where you can add/remove items from both the front and the back efficiently. Used by `Time_RewindManager` as its history buffer.

Enum: A fixed set of named constants. Each constant is a full object. `CharacterConfig` and `BossConfig` are enums.

Game Loop: The cycle of `act()` calls that drives the game forward, ~60 times per second.

HashMap: A Java data structure for looking up values by key. Used by `GenericPlayer` to store animations (`animations.get("Dash")`).

Interface: A contract. A list of method signatures that any implementing class must provide. `GameState`, `Ability`, `Player`, `IActiveGameState`, and `Time_Snapshottable` are interfaces.

Memento Pattern: A design pattern for saving and restoring state. `Time_ActorMemento` is the memento. `Time_Snapshottable` is the originator.

Reflection: Java's ability to inspect and create classes by name at runtime. Used in `GenericPlayer` to load ability classes from string names (`Class.forName("Ability_StandPunch").newInstance()`).

Seeded RNG: A random number generator initialized with a specific starting value (the "seed"). Same seed = same sequence of numbers. Used by `GameRNG` so the rewind system can replay the same obstacle spawn pattern.

Stack: A data structure where the last item added is the first item removed (LIFO — Last In, First Out). The Game State Manager uses a stack to manage states.

Static field/method: Belongs to the class itself, not to any object instance. `ScoreManager.score` is static — there is only one score, not one per `ScoreManager` object.

State Machine: A system where a thing can be in exactly one "state" at a time, and transitions between states in defined ways. The GSM is a state machine for the game's scenes. The `Banner` class is a mini state machine (slideIn → hold → slideOut).

Transparency: In Greenfoot images, 0 = fully invisible, 255 = fully opaque. Used for fading effects and the i-frame blink.

End of Guide — Greenfoto Learning Guide v1.0

Generated to help you understand and take ownership of your own code.